HackSphere 3.0

HackSphere 3.0: Smart Energy Microgrid & Demand Forecasting Engine

1. Problem Overview

Modern power grids are rapidly integrating dynamic, localized renewable energy storage to handle sudden load shifts and reduce blackout risks. Modern microgrids require real-time demand forecasting to switch seamlessly between Main Grid power and local battery storage.

Your challenge is to build an end-to-end web application deployed on **Zoho Catalyst** that forecasts hourly energy demand for the next 24 hours, trains a predictive model using **Zoho Catalyst QuickML**, and triggers automated power-source transition signals between Main Grid power and Renewable Battery Storage.

---

2. Starter Kit Contents (`HackSphere_Starter_Kit.zip`)

To help teams kick-start development, download the official starter kit archive. It contains:

- `PROBLEM_STATEMENT.md`: Copy of the current document in markdown.
- `train_telemetry.csv`: Historical training dataset (80% split) containing weather features and ground-truth energy demand (`target_demand_kw`). Use this file directly in **Zoho Catalyst QuickML** to train your regression pipeline.
- `sample_test_input.csv`: Sample test dataset (20% split) with weather features (omitting `target_demand_kw`) for local code testing and validation.
- `predict.py`: Starter Python template pre-configured to format test inputs, call your published QuickML API endpoint, and export predictions.
- `test_eval.py`: Pre-submission self-test script. Run `python test_eval.py` locally before submitting to ensure your `predict.py` script meets the required contract.

---

3. Key Requirements

1. **Model Building:** Train and publish a demand forecasting regression model inside **Zoho Catalyst QuickML**.
2. **Data Ingestion & Cleanup:** Accept telemetry inputs via a live UI form or CSV uploads. Clean and validate data, handle missing values, and prepare feature vectors matching QuickML expectations.
3. **Demand Forecasting:** Call your published QuickML endpoint to generate hourly demand predictions (kW) for each row in a 24-hour forecast window.
4. **Smart Grid Switching Logic:** Apply threshold and time-based transition rules to predicted values to output real-time power dispatch signals.
5. **Interactive Web Dashboard:** Host a responsive dashboard on Zoho Catalyst displaying predicted load curves, active grid states, battery state-of-charge, and overload alerts.

---

4. Recommended Catalyst Architecture

- **Step-by-Step Onboarding Guide:** Catalyst for HackSphere 3.0
  (Free account registration, services and architecture summary)
- **Zoho Catalyst Platform Demo Video:** Watch Catalyst AI Video
  (Showcasing Deployment ease with Catalyst)

Feature / Module	Catalyst Service	Description
Telemetry Ingestion	**Slate + Stratus**	Build Slate UI forms for manual inputs; store uploaded CSV files in Stratus file storage.
Preprocessing & API	**AppSail / Functions**	Python backend to parse timestamps, compute rolling/lag features, and handle API requests.
Forecasting Engine	**QuickML**	Train, optimize, and deploy the regression pipeline; call its REST endpoint from your app.
Transition Controller	**Functions / AppSail**	Evaluate 24-hour predictions against grid transition rules and emit control signals.

Feature / Module	Catalyst Service	Description
Live Dashboard	**Slate + Data Store / NoSQL**	Render charts, grid state badges, and system logs; persist telemetry in Data Store / NoSQL.

5. Dataset and Telemetry Schema

The starter kit provides `train_telemetry.csv` for training and `sample_test_input.csv` for local testing.

Column	Type	Units / Format	Description
`timestamp`	String	`YYYY-MM-DD HH:MM:SS`	Hourly timestamp for the reading or forecast hour.
`ambient_temp_c`	Float	°C	Outdoor ambient temperature.
`humidity_pct`	Float	%	Relative humidity percentage.
`solar_irradiance_wm2`	Float	$\mathrm{W/m^2}$	Solar power potential.
`target_demand_kw`	Float	kW	Actual energy demand (*Training target only; omitted from test inputs*).

Inference Contract

- Supply one feature row for each forecast hour (24 rows for a full 24-hour forecast).
- Each row must include the timestamp and the three weather features (`ambient_temp_c`, `humidity_pct`, `solar_irradiance_wm2`).
- The QuickML model returns one numerical prediction per row.
- If custom lag/rolling features are created, define them consistently for both training and inference.

6. Transition Controller Logic

Evaluate the rules for each forecast hour using the predicted demand (kW) and timestamp hour (18:00–22:00 window is inclusive):

Condition	Grid Mode / Alert State
Demand is **> 160 kW**, **OR** timestamp hour is between **18:00 and 22:00 inclusive**.	**Renewable Storage Mode**
Otherwise (Demand $\leq$ 160 kW AND outside 18:00–22:00 window).	**Main Grid Mode**
Demand is $\geq$ **200 kW**.	**Trigger Critical Overload Alert** *(independent of grid mode)*

7. Evaluation Criteria

Criteria	Weight	Description
Model Accuracy (QuickML)	**35%**	Measured via Root Mean Squared Error (RMSE) on a hidden test dataset by executing `predict.py` against the team's published QuickML API.
Deployment & Functionality	**35%**	Live, publicly accessible Zoho Catalyst web application URL showing end-to-end workflow execution.
Transition Controller Logic	**20%**	Correct execution of power-source transition signals and overload alerts based on forecast outputs.
UI & User Experience	**10%**	Clean visual dashboard with real-time demand charts, active state badges, and system alert banners.

8. Automated Script (`predict.py`) & Self-Testing

Local Development Test

During development, run `predict.py` locally to verify that it correctly connects to your published QuickML API endpoint and outputs formatted CSV predictions:

```
python predict.py --input sample_test_input.csv --output pred
ictions.csv
```

Expected `predictions.csv` Format:

```
timestamp,target_demand_kw
2024-01-01 00:00:00,128.45
2024-01-01 01:00:00,119.30
```

(Note: Must return exactly one prediction per input row, preserving the original timestamp).

Mandatory Pre-Submission Check (`test_eval.py`)

Before handing in your submission, run the self-testing script:

```
python test_eval.py
```

If this script outputs `SUCCESS`, your `predict.py` contract is valid and safe for automated evaluation.

---

9. Submission Deliverables

Submit your solution before the hackathon deadline with the following deliverables:

1. **Public Catalyst App URL:** Live link to your web application hosted on Catalyst Slate / AppSail.
2. **Project Code Base:** Public GitHub Repository URL **OR** zipped source folder (`TeamName_Submission.zip`).
3. **Required Repository Contents:**
   - Full application code (Frontend & Backend).
   - Working `predict.py` configured with your published QuickML endpoint URL.
   - `README.md` detailing setup steps, architecture overview, and required environment variables (keep QuickML credentials secure).

All the best!